

Sistemi Embedded
per lo
Smart Metering

Indice

1	Introduzione ai Sistemi Embedded	2
1.1	Definizione e Caratteristiche dei Sistemi Embedded	2
1.2	Evoluzione Storica dei Sistemi Embedded	3
1.3	Metodologia di progettazione e vincoli di sistema	5
1.4	Sistemi Real-Time	5
1.5	Distinzione tra Software e Firmware	6
1.6	Microprocessori e Microcontrollori	7
1.7	Architetture Software per Sistemi Embedded	10
1.8	Prototipazione rapida e ingegnerizzazione	13
2	Fondamenti della programmazione in C	15
2.1	Architettura della Toolchain	15
2.1.1	Runtime Startup Code e Avvio del Sistema	17
2.2	Struttura del programma e sintassi fondamentale	18
2.2.1	Tipizzazione dei dati e dichiarazione delle variabili	20
2.3	Organizzazione della memoria e classi di memorizzazione	21
2.4	Puntatori	22

Capitolo 1

Introduzione ai Sistemi Embedded

Questo capitolo introduce i concetti fondamentali, l'evoluzione storica e i vincoli di progettazione dei sistemi embedded. Vengono approfondite le differenze architetturali tra microprocessori e microcontrollori, analizzando il ruolo discriminante della MMU e la gerarchia delle memorie. Infine, vengono esaminati i principali paradigmi software, dai modelli Foreground-Background ai sistemi operativi Real-Time e Linux Embedded, fino ai requisiti di ingegnerizzazione per la transizione dal prototipo al prodotto finito.

1.1 Definizione e Caratteristiche dei Sistemi Embedded

Un sistema embedded è un dispositivo di calcolo a microprocessore o microcontrollore integrato all'interno di un sistema fisico più ampio, progettato per svolgere un'attività specifica o un insieme ben definito di compiti strettamente correlati. A differenza dei sistemi di calcolo General-Purpose, l'utente finale non ha la facoltà di installare, rimuovere o modificare le applicazioni software.

I sistemi informatici si dividono nelle seguenti categorie:

- **Sistemi General-Purpose** → calcolatori destinati a un'ampia classe di applicazioni eterogenee, programmabili dall'utente finale e orientati a massimizzare la velocità media di calcolo secondo il criterio per cui una maggiore velocità corrisponde a un sistema migliore (es. personal computer, laptop, server, mainframe).
- **Sistemi Embedded** → dispositivi monofunzionali o a funzionalità dedicata, integrati all'interno dell'ambiente di lavoro con minimo intervento dell'operatore.
- **Sistemi Real-Time** → piattaforme in cui la correttezza della risposta è vincolata al rispetto di scadenze temporali certe, non necessariamente coincidenti con sistemi embedded.
- **Sistemi Embedded Real-Time** → sistemi embedded dedicati operanti sotto vincoli temporali stringenti di reazione e determinismo.

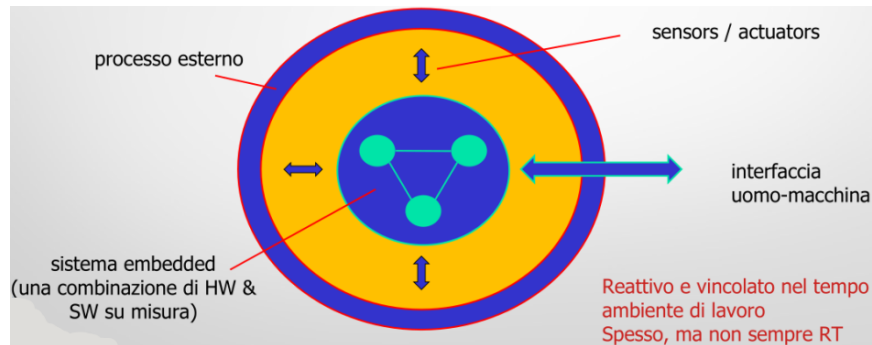
I dispositivi convenzionali quali personal computer e smartphone non costituiscono device embedded in quanto non sono dedicati a una singola attività, permettono l'esecuzione contemporanea o sequenziale di molteplici programmi applicativi a discrezione dell'utente e presentano un'interazione utente continua e non predefinita.

I sistemi embedded sono caratterizzati da specifici attributi architetturali e funzionali:

- **Monofunzionalità** → esecuzione ripetitiva di una singola funzione specifica o di un insieme molto ristretto di attività correlate, ottimizzate sulle peculiarità dell'hardware.
- **Interazione minima con l'utente** → operatività automatizzata con interventi ridotti o assenti da parte dell'operatore umano.

- **Vincoli progettuali stringenti** → requisiti rigidi in termini di costo ridotto (low cost), consumo energetico minimo (low power), ingombro dimensionale compatto (small size) e semplicità di utilizzo.
- **Operatività continua e non presidiata** → funzionamento prolungato nel tempo (24/7) senza necessità di supervisione, con elevatissimi standard di affidabilità e robustezza.

Dal punto di vista funzionale, il sistema embedded costituisce una *combinazione su misura di componenti hardware e software reattiva e vincolata nel tempo rispetto all'ambiente operativo circostante*. Il dispositivo riceve informazioni dal *processo esterno* attraverso una serie di *sensori*, elabora i dati acquisiti e agisce direttamente sull'ambiente tramite *attuatori*, interfacciandosi all'occorrenza con un'*interfaccia uomo-macchina*.

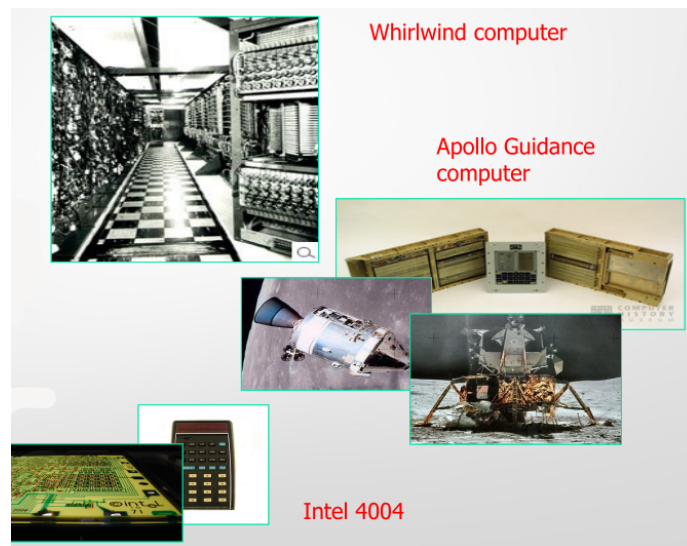


I campi di applicazione dei sistemi embedded comprendono molteplici settori dell'ingegneria e dell'industria:

- **Elettronica di consumo** → lettori MP3, altoparlanti wireless, auricolari, dispositivi indossabili, periferiche per personal computer, telecomandi, ricevitori televisivi non smart, accessori dedicati per smartphone.
- **Elettrodomestici** → forni a microonde, lavatrici, frigoriferi, impianti di condizionamento, caldaie, centraline di sicurezza domestica.
- **Automazione d'ufficio** → apparati di connessione modem, stampanti, scanner.
- **Attrezzature aziendali** → impianti di allarme antintrusione, terminali per punti vendita (POS).
- **Automotive** → centraline di iniezione elettronica, sistemi di frenata antibloccaggio.
- **Avionica, Aerospazio e Trasporti** → sistemi di navigazione satellitare e inerziale, strumentazione di bordo, piattaforme Digital Fly-By-Wire per aeromobili, treni e sottomarini.
- **Medicina** → sistemi di supporto vitale, dispositivi medici impiantabili, strumentazione di diagnostica clinica e monitoraggio, apparati di telemedicina.
- **Telecomunicazioni** → commutatori di rete, router, apparati di ricetrasmisione modem LTE.
- **Automazione industriale** → sistemi di monitoraggio ambientale e di processo, manipolatori robotici e controllori di asse.

1.2 Evoluzione Storica dei Sistemi Embedded

L'origine dei sistemi dedicati risale alla fine degli anni '40 con lo sviluppo del calcolatore MIT Whirlwind, espressamente concepito per la simulazione del comportamento dinamico e delle reazioni di volo di un aeromobile in tempo reale.

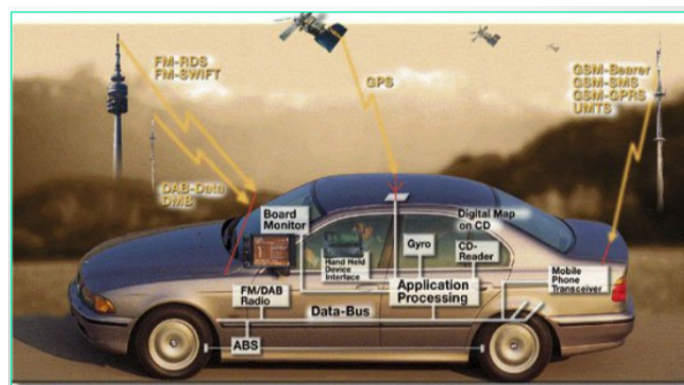


Negli anni '60 venne realizzato il primo sistema embedded universalmente riconosciuto: l'Apollo Guidance Computer, progettato dal MIT Instrumentation Laboratory per gestire in autonomia la navigazione, il calcolo della traiettoria e il controllo del modulo di comando e del modulo lunare del programma Apollo.

Alla fine degli anni '80, con l'avvento dei microprocessori a singolo circuito integrato (inaugurati dal componente Intel 4004), i sistemi embedded hanno smesso di rappresentare un'eccezione tecnologica per trasformarsi nello standard operativo della quasi totalità degli apparati elettronici.

Nei sistemi attuali, l'architettura embedded ha assunto una struttura complessa basata su reti eterogenee di processori distribuiti, in grado di eseguire sistemi operativi standard e di comunicare attraverso bus cablati (come i bus di campo industriali e automobilistici), reti wireless, reti LAN e WAN. All'interno di un'automobile moderna di fascia alta possono essere presenti fino a circa 100 processori dedicati, ripartiti gerarchicamente per compiti:

- Microcontrollori a 4 bit dedicati a funzioni ausiliarie elementari, come il blocco e il pretensionamento delle cinture di sicurezza.
- Microcontrollori a 16 bit preposti alla gestione degli strumenti e degli indicatori del cruscotto.
- Microcontrollori o processori a 16 e 32 bit per la centralina di controllo motore (ECU).
- Processori a 32 e 64 bit dedicati all'elaborazione grafica e multimediale dei sistemi di infotainment di bordo.



1.3 Metodologia di progettazione e vincoli di sistema

La progettazione di un sistema embedded adotta un approccio flessibile e iterativo, in cui le fasi preliminari di specifica dei requisiti avvengono spesso contemporaneamente alla stesura dell'architettura hardware e software:

1. **Analisi dei requisiti** → identificazione dettagliata del contesto operativo in cui il dispositivo deve operare, definizione delle funzionalità da svolgere e degli obiettivi prestazionali richiesti.
2. **Codesign Hardware/Software** → ripartizione delle funzioni tra silicio ed elaborazione logica, selezione della piattaforma di calcolo (microcontrollore o microprocessore in funzione di frequenza di clock, memoria e MIPS necessari), identificazione del corredo di periferiche e scelta dell'ambiente software (architettura Foreground-Background, RTOS minimale o sistema operativo completo).

Essendo progettato per essere special-purpose, un sistema embedded deve garantire la massima efficienza nell'esecuzione continua delle sue specifiche mansioni, rispettando rigorosi vincoli costruttivi:

- Elevata efficienza energetica con minimizzazione dei consumi (low power), fondamentale per apparati alimentati a batteria o privi di manutenzione.
- Costi computazionali contenuti, evitando sovradimensionamenti hardware ingiustificati.
- Basso footprint di memoria (quantità di memoria che un programma utilizza durante la sua esecuzione), con allocazione ottimizzata sia per il codice di programma sia per le strutture dati.
- Ridotto ingombro dimensionale e peso limitato, requisiti imprescindibili per applicazioni inserite all'interno di dispositivi portatili, veicoli o apparati medici quali i pacemaker.

L'affidabilità (reliability) costituisce un parametro cardine di valutazione, misurato quantitativamente attraverso due metriche fondamentali:

- **MTTF** (Mean Time To Failure) → tempo medio intercorrente fino al manifestarsi di un guasto permanente, che deve essere massimizzato.
- **MTTR** (Mean Time To Repair) → tempo medio necessario alla riparazione o ripristino della piena operatività, che deve essere minimizzato.

Il sistema deve inoltre soddisfare requisiti di sicurezza declinati nella duplice dimensione di:

- **Safety** → protezione da guasti accidentali, rischi operativi e malfunzionamenti fortuiti, garantendo che il dispositivo non arrechi in alcuna condizione danni fisici agli utenti o all'ambiente circostante.
- **Security** → protezione attiva contro minacce informatiche e attacchi deliberati, assicurando la riservatezza, l'integrità delle informazioni e l'accesso ai soli soggetti o processi esplicitamente autorizzati.

1.4 Sistemi Real-Time

Nei sistemi embedded, l'affidabilità è strettamente correlata alla **reattività**, definita come l'attitudine del sistema a elaborare e rispondere agli stimoli provenienti dall'ambiente esterno entro finestre temporali precise.

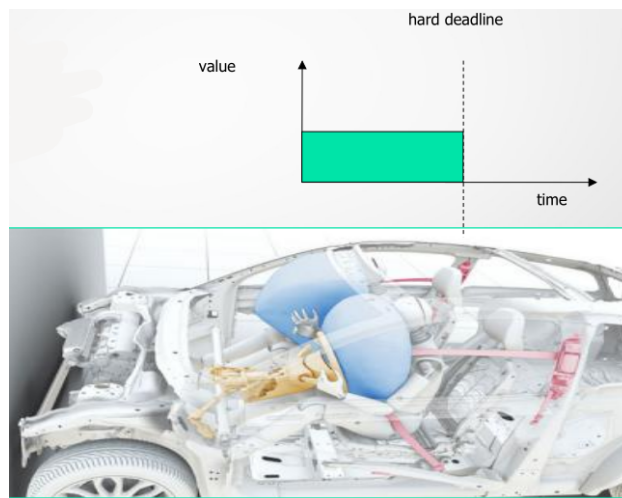
Il termine Real-Time trae la propria origine dalle prime simulazioni di volo e di processi complessi, in cui la simulazione procedeva a una velocità coincidente con quella dell'evento fisico reale simulato. In base al comportamento temporale, i sistemi informatici si distinguono in tre macro-categorie:

- **Sistemi Batch** → elaborazioni in cui non sussiste alcuna scadenza operativa vincolante tra l'insorgenza dell'evento e la produzione della risposta (es. esecuzione di una stored procedure SQL).
- **Sistemi Interactive On-line** → contesti in cui un tempo di risposta rapido è auspicabile per migliorare l'usabilità, ma non condiziona la correttezza logica del risultato (es. sistemi operativi desktop convenzionali).
- **Sistemi Real-Time** → sistemi in cui la risposta deve essere erogata inderogabilmente entro un intervallo temporale prestabilito, pena il fallimento del sistema stesso.

Un sistema Real-Time non è necessariamente un sistema embedded. In relazione alla severità del vincolo temporale (deadline), si individuano due tipologie di funzionamento:

- **Hard Real-Time** → il rispetto della scadenza temporale (deadline) è assoluto e tassativo. Il mancato completamento del calcolo o dell'azione entro il limite stabilito equivale al collasso del sistema e può comportare danni catastrofici a beni, operatori o ambiente circostante. Tipicamente associati a una complessità software contenuta, i sistemi Hard Real-Time non sono semplicemente sistemi veloci, bensì sistemi certificati per garantire tempi di intervento deterministici e certi.
- **Soft Real-Time** → la presenza di una finestra di tolleranza ammette ritardi occasionali nella risposta; il superamento della scadenza non pregiudica l'incolumità delle persone né l'integrità dell'ambiente, comportando unicamente un degrado delle prestazioni fornite.

Un esempio primario di applicazione Hard Real-Time è costituito dal sistema di attivazione dell'airbag nei veicoli a motore, in cui il rilevamento della collisione e il comando di innesco devono avvenire rigorosamente entro una frazione di millisecondo prefissata.



1.5 Distinzione tra Software e Firmware

Il concetto di firmware è intrinsecamente legato al settore dei sistemi embedded e dei microcontrollori. Il termine identifica la frazione del codice software posizionata a più stretto contatto con l'hardware fisico sottostante.

Più formalmente, il firmware è l'insieme delle istruzioni residenti permanentemente all'interno della memoria non volatile del dispositivo, preposte a inizializzare, controllare e alimentare l'hardware specifico destinato all'applicazione.

Sul piano pratico e metodologico si individua una netta demarcazione di impiego:

- **Software** → associato all'ambiente di calcolo General-Purpose, caratterizzato da livelli di astrazione elevati, portabilità tra diverse macchine e gestione tramite sistema operativo.

- **Firmware** → associato all'architettura dei microcontrollori, sviluppato su misura per pilotare direttamente registri e periferiche della piattaforma embedded senza intermediari astratti.

Tipicamente su un microcontrollore non girano dei software, ma è possibile farlo in alcuni casi. Ad esempio, quando si necessitano librerie grafiche o di rete si può caricare un sistema operativo sul microcontrollore per evitare di scrivere manualmente dei driver.

1.6 Microprocessori e Microcontrollori

Le unità di elaborazione utilizzate nell'elettronica digitale comprendono diverse famiglie di dispositivi:

- **General-Purpose Processors (GPP o CPU)** → unità centrali di elaborazione destinate all'informatica generale e a compiti computazionali complessi.
- **Microcontrollori (MCU)** → dispositivi integrati su singolo chip orientati al controllo diretto di sistemi fisici e periferiche.
- **Digital Signal Processor (DSP)** → processori specializzati nell'elaborazione numerica real-time di segnali analogici digitalizzati.
- **Advanced RISC Machine (ARM)** → famiglie di processori basati su architettura RISC, scalabili da microcontrollori a processori applicativi ad alte prestazioni.

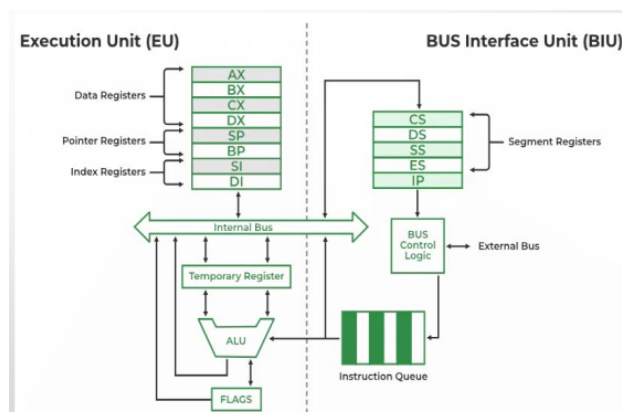
Microprocessore General-Purpose

Un microprocessore General-Purpose (GPP / MPU) che presenta le seguenti caratteristiche:

- **Architettura Von Neumann** → dati ed istruzioni condividono la stessa memoria e lo stesso bus.
- **Set di istruzioni CISC** (Complex Instruction Set Computer) → capaci di svolgere più operazioni in un'unica istruzione. Un microprocessore moderno utilizza spesso anche tecnologia RISC (Reduced Instruction Set Computer → set ridotto di istruzioni semplici).

È costituito al suo interno da:

- Una ALU (Arithmetic Logic Unit) dedicata all'esecuzione delle operazioni aritmetiche e logiche.
- Un banco di registri interni per la memorizzazione temporanea dei dati e la gestione delle istruzioni.
- Un bus interno ad alta velocità.
- Circuiti di controllo e di temporizzazione per la sincronizzazione delle operazioni.
- Un'interfaccia verso memorie esterne ad alta velocità.



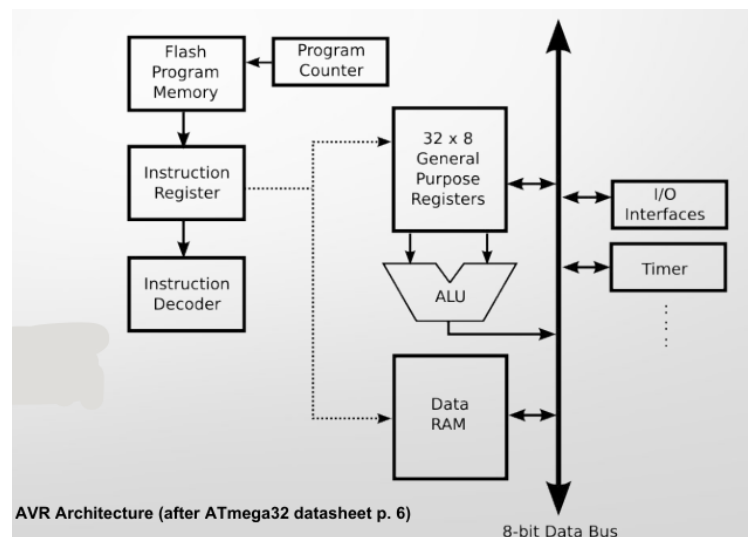
Per funzionare correttamente, un microprocessore necessita di componenti e risorse circuitali esterne al chip:

- Un'ampia memoria volatile di lavoro (RAM esterna) e supporti di memoria di massa (hard disk o memorie a stato solido) per la gestione di grandi quantità di dati.
- Interfacce e controller dedicati per periferiche di ingresso (tastiere, puntatori, scanner, microfoni).
- Interfacce per dispositivi di uscita (display, monitor, stampanti, attuatori audio).
- Potenza di alimentazione elevata, con consumi che possono raggiungere decine o centinaia di watt.

Microcontrollore

Un microcontrollore (MCU) è un dispositivo di elaborazione dati concettualmente analogo al microprocessore ma basato tipicamente su **architettura Harvard** (con bus e spazi di memoria separati per istruzioni e dati) e **set di istruzioni RISC**. L'elemento distintivo fondamentale risiede nell'integrazione monolitica su un unico circuito integrato (single-chip) di tutti i sottosistemi necessari all'elaborazione:

- Una CPU RISC.
- Una memoria di programma non volatile interna (EPROM, EEPROM o Flash).
- Una memoria volatile di lavoro interna (SRAM di dimensioni limitate, da pochi kB a centinaia di kB).
- Porte digitali di ingresso e uscita (GPIO).
- Timer, contatori hardware e convertitori analogico-digitali (ADC).
- Interfacce di comunicazione seriale sincrone e asincrone (UART, SPI, I2C, I2S, RMII, generatori PWM).



Mentre i microprocessori costituiscono il nucleo di personal computer, cluster e server, i microcontrollori sono il fondamento dei dispositivi industriali, autronici e domotici. La MCU include sia i componenti base di una CPU (seppure con capacità ridotta) sia una serie di periferiche hardware che il microprocessore non possiede.

Il confronto strutturale evidenzia precise divergenze progettuali:

- **Microcontrollore (MCU):** architettura Harvard e RISC, memorie RAM e Flash integrate internamente sul die, consumi energetici ridotti, basso costo unitario, elevata densità di periferiche on-chip, dotazione hardware nativa per il determinismo Real-Time (controller DMA dedicati, matrici di bus interne senza contesa, gestione interrupt vettorizzata con livelli multipli di priorità e nesting).
- **Microprocessore (MPU):** architettura Von Neumann e prevalentemente CISC, memorie RAM e ROM collocate esternamente rispetto al die, frequenze di clock elevate, elevatissima potenza di calcolo computazionale, ridotto numero di periferiche integrate on-chip, throughput elevato verso la RAM esterna, molteplici livelli di cache integrata e presenza della MMU.

Nelle attuali generazioni di silicio le barriere architetturali tendono a sfumare: esistono microcontrollori avanzati con capacità computazionali assimilabili a microprocessori, microprocessori arricchiti con periferiche integrate (ADC e timer), microcontrollori dotati di cache o bus di espansione verso memorie esterne, e architetture eterogenee multicore (HMP - Heterogeneous Multi-core Processing) che integrano core applicativi MPU e core MCU dedicati al Real-Time sul medesimo silicio.

L'elemento cardine che traccia la reale linea di demarcazione tra microprocessori e microcontrollori è la presenza o l'assenza della **MMU (Memory Management Unit)**.

La MMU costituisce il blocco circuitale indispensabile per consentire l'esecuzione di un sistema operativo General-Purpose moderno. Le sue funzioni primarie includono:

- **Virtualizzazione degli indirizzi** → conversione degli indirizzi di memoria virtuali generati dai processi software negli effettivi indirizzi fisici della memoria RAM.
- **Protezione della memoria** → impedisce a singoli processi di accedere senza autorizzazione alla memoria di altri processi o allo spazio riservato del kernel.
- **Gestione della frammentazione** → ricomposizione logica di pagine allocate in modo frammentato nella memoria fisica, garantendo allo spazio utente la visibilità di un'area di memoria perfettamente contigua e massimizzando lo sfruttamento dello spazio.
- **Controllo della cache** → supervisione delle politiche di allocazione della memoria cache e mantenimento della coerenza dei dati tra cache e memoria centrale.
- **Arbitraggio del bus** → regolamentazione e prioritizzazione degli accessi concorrenti ai bus dati e indirizzi tra CPU e periferiche master.
- **Gestione dei banchi di memoria** → coordinamento e selezione dinamica dei differenti moduli e banchi costituenti il sottosistema di memoria.

Nei microcontrollori tradizionali la MMU è assente, poiché il meccanismo di virtualizzazione e rilocazione delle pagine introduce latenze non prevedibili, incompatibili con i requisiti di determinismo temporale tipici dei sistemi embedded e Real-Time.

Un'altra differenza fondamentale tra microprocessori e microcontrollori è sulle **taglie di memoria**, che differiscono sistematicamente di almeno due ordini di grandezza:

- **Memoria Volatile (RAM):**
 - MCU RAM interna → da pochi kB fino a un limite massimo di circa 1 MB, con valori tipici compresi nell'ordine delle centinaia di kB.
 - MCU RAM esterna → moduli opzionali espandibili da 64 MB fino a 512 MB o 1 GB.
 - MPU RAM → configurazioni standard su banchi esterni pari a 16 GB o 32 GB.

- **Memoria Non Volatile (Flash e Storage):**

- MCU Flash interna → da poche decine di kB fino a 2 MB per la memorizzazione permanente del firmware.
- MCU Flash esterna → memorie collegate tramite interfacce ad alta velocità (es. QSPI) estendibili fino a qualche GB.
- MPU Mass Storage → memorie di massa secondarie con capacità nell'ordine di 2 TB o superiori.

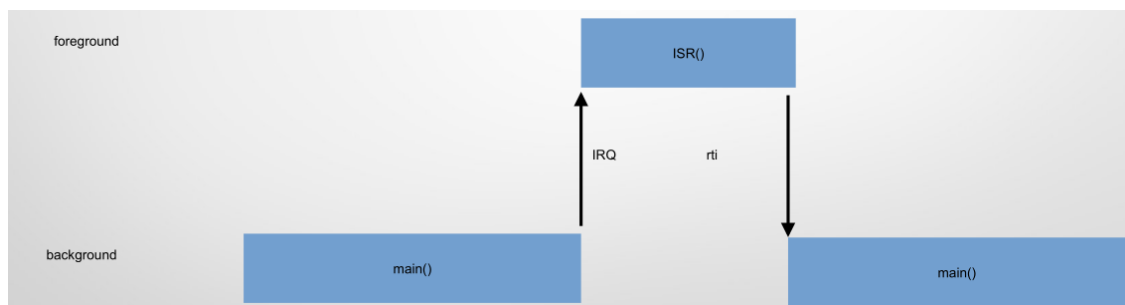
1.7 Architetture Software per Sistemi Embedded

La realizzazione del software nei sistemi embedded si fonda su tre principali paradigmi operativi: il modello ciclico Foreground-Background, i sistemi operativi Real-Time (RTOS) e le distribuzioni Linux Embedded.

Foreground-background

Nel modello Foreground-Background (superloop con interruzioni), l'esecuzione è strutturata su due livelli di priorità:

- **Background** → costituito dalla funzione ciclica `main()`, un ciclo infinito in cui vengono eseguite in sequenza le attività a bassa priorità, le elaborazioni non critiche e la manutenzione ordinaria dello stato del sistema.
- **Foreground** → costituito dalle Routine di Servizio delle Interruzioni (**ISR**), richiamate asincronamente al manifestarsi di segnali di interrupt hardware (**IRQ**) generati dalle periferiche o dall'esterno. Le ISR sospendono l'esecuzione del ciclo di background per gestire immediatamente gli eventi prioritari e urgenti, restituendo quindi il controllo al punto interrotto mediante l'istruzione di ritorno da interruzione (**rti**).

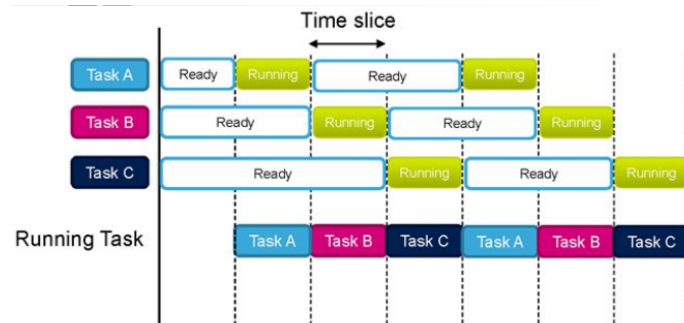


Sistemi operativi Real-Time (RTOS)

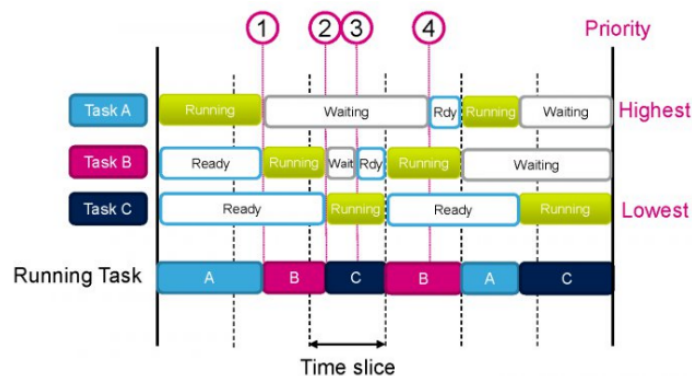
Nei contesti a complessità superiore si impiegano gli RTOS (Real-Time Operating System), sistemi operativi minimali a bassissimo footprint di memoria sviluppati appositamente per microcontrollori. Le loro peculiarità includono:

- Assenza della gestione dinamica della memoria, evitata per scongiurare fenomeni di frammentazione dell'heap e garantire tempi di allocazione deterministici.
- Presenza di politiche di scheduling tassativamente concepite per garantire la certezza dei tempi di esecuzione dei task.

- Meccanismi di arbitraggio dell'elaborazione basati su:
 - **Scheduling cooperativo Round Robin** → suddivisione equa del tempo di CPU tra processi di pari livello.



- **Scheduling preemptive Priority-based** → prelazione immediata della CPU a favore del task a priorità superiore.

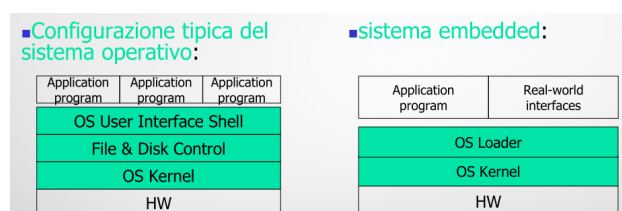


NOTA: nelle applicazioni embedded si usa spesso, in modo improprio, il termine **task**. In realtà si tratta di **thread**, non di task. Entrambi indicano flussi di operazioni, ma con una differenza fondamentale: i thread condividono la stessa memoria, mentre i task no. Nei sistemi basati su RTOS, quindi, il termine corretto è thread.

Linux Embedded

L'architettura software di un sistema operativo embedded differisce radicalmente da quella di un sistema operativo convenzionale General-Purpose:

- Nel **sistema operativo General-Purpose**, al di sopra del kernel e del gestore del caricamento (OS Loader) risiedono componenti pesanti quali il file system, il controllo del disco e l'interfaccia utente grafica (OS User Interface Shell), preposti a lanciare ed eseguire una moltitudine di programmi applicativi indipendenti.
- Nel **sistema operativo embedded**, l'architettura è snella: al di sopra dell'hardware e del kernel risiedono esclusivamente l'OS Loader e i moduli di interfacciamento con il mondo fisico reale (Real-world interfaces), direttamente asserviti alla singola applicazione dedicata.



Per sistemi basati su System-on-Chip (SoC) o Application Processor complessi trova impiego Linux Embedded. Trattandosi di un sistema operativo open-source universale sostenuto da un'ampia comunità internazionale e dal settore industriale, Linux consente di configurare e compilare su misura sia il kernel sia l'intera distribuzione, eliminando tutti i servizi non necessari e riducendo l'impronta di memoria ai minimi requisiti dell'applicazione.

Il principale beneficio dell'approccio Linux risiede nella possibilità di integrare agevolmente moduli e librerie software open-source preesistenti, accelerando lo sviluppo applicativo. Tuttavia, il kernel Linux convenzionale non presenta caratteristiche Real-Time native; per ottenere determinismo si ricorre alla patch Real-Time di Linux (PREEMPT_RT), estesamente adottata in contesti avanzati di robotica e intelligenza artificiale. Linux Real-Time assicura un comportamento rigorosamente prevedibile, garantendo che le attività a priorità critica ricevano immediata attenzione ed esecuzione entro scadenze temporali vincolanti. Sistemi operativi commerciali proprietari per PC (come Windows) risultano invece inadatti per applicazioni embedded critiche a causa della suscettibilità a blocchi sistemici imprevisti (es. schermate blu di errore / BSOD) e dell'assenza di determinismo temporale.

I sistemi operativi Linux embedded trovano largo impiego nel mondo degli apparati di rete (router, access point, ecc...) con *OpenWrt*, bilance elettroniche, TV box (*Dreambox*), automazione industriale e acquisizione dati (*National Instruments CompactRIO*), domotica e smart home (*Google Nest*), gaming e console portatili (*Nintendo Switch*) e tanto altro ancora.

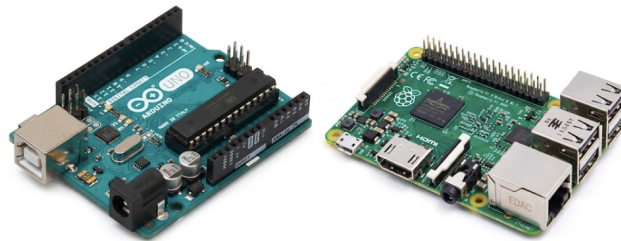


1.8 Prototipazione rapida e ingegnerizzazione

L'evoluzione dell'elettronica digitale ha reso la tecnologia contemporaneamente sempre più complessa e sempre più accessibile. Piattaforme hardware modulari di Rapid Prototyping hanno abilitato la nascita del movimento dei **Maker**, consentendo a progettisti e hobbisti di sperimentare soluzioni innovative e introdurre rapidamente l'elettronica in ambiti inediti.

Le piattaforme di prototipazione rapida maggiormente diffuse si ripartiscono in due categorie tecnologiche distinte:

- **Arduino** → soluzione embedded incentrata su firmware puro, basata su architettura a microcontrollore e orientata al controllo diretto di I/O fisici.
- **Raspberry Pi** → soluzione Linux embedded incentrata su microprocessore e architettura SoC, orientata ad applicazioni complesse ad alto livello di astrazione.



Sebbene l'uso di piattaforme rapide faciliti lo sviluppo iniziale grazie alla disponibilità di librerie, driver e schede di espansione preassemblate, sussiste una differenza fondamentale tra un approccio hobbistico o amatoriale e un approccio ingegneristico professionale:

- Un **prototipo** (Proof of Concept) rappresenta una validazione concettuale di laboratorio. Non è progettato per durare nel tempo e opera unicamente nelle condizioni ambientali migliori possibili (Best Case).
- Un **prodotto finito** deve essere interamente ingegnerizzato, rispettando la totalità dei requisiti di affidabilità, compattezza, sicurezza e conformità normativa fissati per i sistemi embedded.

L'ambiente reale in cui il prodotto embedded è chiamato a operare è intrinsecamente severo e ostile. Un'ingegnerizzazione rigorosa impone il passaggio fondamentale dalla progettazione nominale alla progettazione per il caso peggiore (Worst Case Design), fronteggiando molteplici condizioni avverse:

- Escursioni termiche ambientali estreme.
- Presenza di contaminanti atmosferici, polvere e particolato.
- Umidità elevata e condense corrosive.
- Sollecitazioni meccaniche continuative e vibrazioni.
- Interferenze elettromagnetiche (EMI/EMC) e disturbi indotti sulle linee di segnale e alimentazione.
- Utilizzo improprio, manomissione o errori da parte dell'utente finale.

La transizione verso un prodotto professionale richiede la piena padronanza dei fondamenti teorici e dei limiti fisici del mondo reale:

- I componenti fisici presentano **limiti operativi stringenti**; ad esempio, la lunghezza dei collegamenti via cavo (cable length) incide su attenuazione, cadute di tensione, ritardi di propagazione e accoppiamenti parassiti.

- Obbligo di conformità e rispetto delle **norme tecniche**, degli standard industriali e delle direttive legislative e di sicurezza vigenti.
- Svolgimento degli iter di collaudo e acquisizione delle **certificazioni obbligatorie** di prodotto.

La sottovalutazione dei vincoli ambientali, l'impiego di prototipi non ingegnerizzati o la presenza di difetti di progettazione nell'hardware e nel firmware possono determinare guasti disastrosi. Nel contesto dei sistemi embedded industriali e mission-critical, un piccolo errore progettuale comporta costi economici estremamente elevati e rischi inaccettabili per la sicurezza.

Capitolo 2

Fondamenti della programmazione in C

I linguaggi di programmazione si distinguono in base alla modalità di traduzione ed esecuzione del codice da parte dell'hardware:

- **Linguaggi compilati** → la traduzione del codice sorgente genera direttamente codice macchina (codice nativo), costituito da istruzioni binarie eseguibili direttamente dalle reti combinatorie del processore (es. C, C++, Rust). L'eseguibile binario viene processato dalla CPU in modo autonomo, senza richiedere la presenza del sorgente. La compilazione è un'operazione preventiva e computazionalmente onerosa, che impone la ricompilazione dell'intero progetto a ogni modifica del sorgente; l'eseguibile risulta hardware-specific e caratterizzato da bassa portabilità, ma assicura massima efficienza computazionale ed esecuzione deterministica idonea per applicazioni real-time.
- **Linguaggi interpretati** → il codice sorgente viene tradotto ed eseguito a run-time mediante un interprete o un ambiente di esecuzione dedicato (es. C#, Visual Basic, Java, Python, MATLAB, LabVIEW). Possono generare un codice intermedio (bytecode) eseguito da una macchina virtuale (come .NET Framework, Java Virtual Machine o Dalvik Machine) oppure essere interpretati istruzione per istruzione. Garantiscono elevata portabilità e consentono modifiche a run-time senza ricompilazione, ma presentano prestazioni inferiori e totale assenza di determinismo temporale, precludendone l'impiego in contesti real-time.
- **Linguaggi di scripting** → interpretano direttamente il testo sorgente inviato all'hardware senza generazione di bytecode intermedio (es. PHP, JavaScript, Bash, Dash). L'interprete si basa su un analizzatore lessicale (lexer), comportando una velocità di esecuzione ulteriormente ridotta a fronte della massima portabilità del codice.
- **Linguaggio Assembly** → fornisce una rappresentazione mnemonica con corrispondenza uno-a-uno rispetto al linguaggio macchina; non richiede processi complessi di astrazione né la presenza di un interprete.

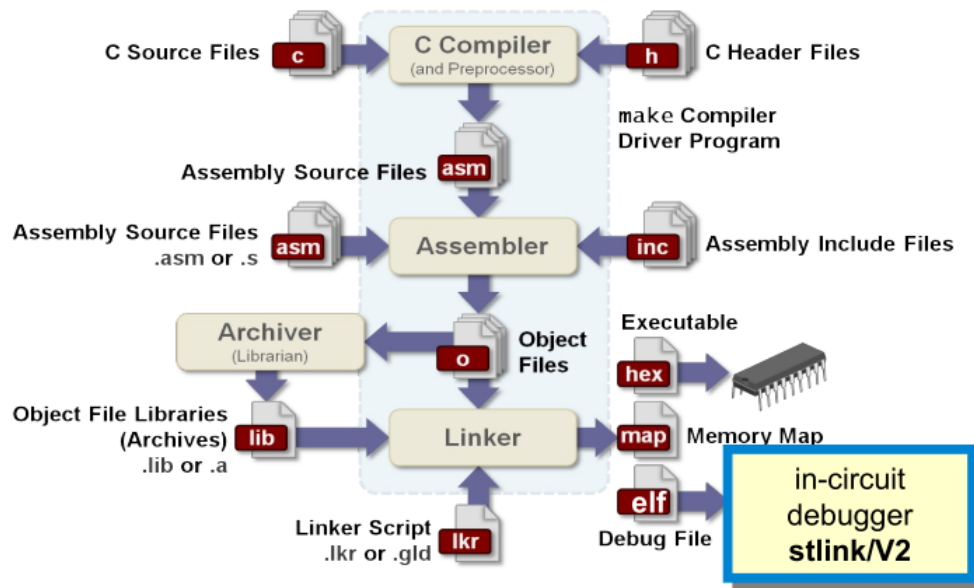
Un caso peculiare tra i linguaggi interpretati è rappresentato dall'ambiente LabVIEW, il quale presenta una doppia interpretazione: il suo interprete è sviluppato in C# (a sua volta linguaggio interpretato con runtime engine) e fa uso di DLL (*Dynamic Link Library*, note come *shared objects* in altri sistemi operativi), ovvero moduli di codice nativo precompilato collegati dinamicamente all'applicazione in fase di esecuzione, in contrasto con il linking statico eseguito al momento della compilazione.

2.1 Architettura della Toolchain

La toolchain rappresenta la suite integrata di strumenti software deputata alla trasformazione del codice sorgente in firmware eseguibile. La configurazione della toolchain è strettamente *hardware-specific*: ogni variazione del processore di destinazione richiede un compilatore, un assembler e

un linker dedicati per la specifica architettura. L'intero flusso è tipicamente coordinato da un programma supervisore (*make* o compiler driver).

Il processo si articola in fasi sequenziali ben definite:



1. **Preprocessing e Compilazione C** → il *preprocessore* espande le macro definite tramite la direttiva `#define` e include il contenuto dei file di intestazione specificati con `#include`. Il compilatore esegue l'analisi lessicale e sintattica del codice sorgente (.c) e genera i corrispondenti file sorgente in linguaggio assembly (.asm).
2. **Assemblaggio** → l'*assembler* elabora i file assembly (.asm, .s) e gli eventuali file ausiliari di inclusione (.inc), traducendoli in file oggetto (.o). I file oggetto contengono codice macchina non ancora consolidato, caratterizzato da riferimenti simbolici non risolti. Per ogni file sorgente C corrisponde un singolo file oggetto; in questa fase possono operare più compilatori e assembler in parallelo.
3. **Archiviazione delle librerie** → tramite il programma archiver (*librarian*), i file oggetto possono essere raggruppati in librerie di archivi statici (.a) o dinamici (.lib).
4. **Linking** → un unico *linker* riceve la totalità dei file oggetto (.o), le librerie di sistema o utente (.lib, .a) e le direttive del Linker Script (.lkr o .gld). Il linker unifica i moduli, associa gli indirizzi fisici effettivi a tutti i simboli (funzioni e variabili globali/esterne) e colloca le sezioni di memoria secondo la mappa dell'architettura.

I prodotti generati al termine del linking comprendono:

- **File Eseguitibile (.hex)** → immagine binaria destinata alla scrittura fisica all'interno della memoria del microcontrollore.
- **Memory Map (.map)** → documento descrittivo recante la ripartizione dettagliata e la collocazione degli indirizzi di memoria assegnati alle sezioni e ai simboli.
- **File di Debug (.elf)** → formato che include il codice eseguibile integrato con la tabella dei simboli e le informazioni di debug, destinato agli strumenti di ispezione.

Il debugger (*in-circuit debugging*) si avvale di una combinazione di elementi hardware e software interfacciati al microcontrollore tramite sonde dedicate (come il modulo hardware ST-Link/V2).

L'analisi degli errori generati durante il flusso consente di isolare la fase di generazione responsabile:

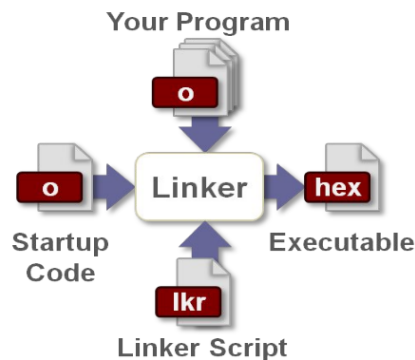
- **Syntax error** → errore rilevato dal compilatore dovuto a violazioni della sintassi formale nel codice sorgente C.

- **Unresolved symbol** → errore segnalato dal linker causato dall'impossibilità di risolvere l'indirizzo di una funzione o di un identificatore (a causa di nomi errati, funzioni non implementate, moduli omessi in compilazione o librerie mancanti).

2.1.1 Runtime Startup Code e Avvio del Sistema

Il software caricato all'interno della memoria di un microcontrollore non è formato solo dal codice scritto dall'utente o generato da configuratori grafici (quali STM32CubeMX), ma include il codice di **runtime C** e il **runtime startup code**:

- **C Runtime (CRT)** → insieme di routine a basso livello e funzioni di supporto necessarie per il corretto funzionamento del linguaggio C su piattaforma embedded; viene collegato automaticamente in fase di linking e si presenta tipicamente sotto forma di sorgente assembly (`crt0.s`) o codice oggetto (`crt0.o`).
- **Runtime Startup Code** → viene eseguito dal processore immediatamente all'uscita dal reset, prima dell'ingresso nella funzione `main()`. Esso è interamente modificabile dallo sviluppatore qualora siano richieste riconfigurazioni dell'ambiente di esecuzione.



Le operazioni eseguite dal runtime startup code sono:

1. Definizione e inizializzazione della tabella dei vettori di interruzione.
2. Configurazione degli handler per la gestione delle eccezioni hardware e software (condizioni anomale quali divisioni per zero o tentativi di accesso non valido in memoria).
3. Gestione del vettore di reset: all'indirizzo fisico 0 della memoria risiede il puntatore all'istruzione di reset, la quale provvede ad azzerare il registro Program Counter (PC), indirizzandolo alla prima istruzione eseguibile.
4. Inizializzazione fisica di tutte le sezioni di memoria dati (copia dei valori iniziali delle variabili dalla memoria non volatile alla RAM e azzeramento dell'area BSS).
5. Caricamento e inizializzazione del registro Stack Pointer e delimitazione dello spazio riservato all'Heap.
6. Salto incondizionato alla prima istruzione della funzione `main()`.

2.2 Struttura del programma e sintassi fondamentale

Un programma in C destinato a sistemi embedded presenta una struttura modulare governata da direttive e blocchi funzionali:

```
1  #include <stdio.h>
2  #define PI 3.14159
3
4  int main(void){
5      float radius, area;
6      //Calculate area of circle
7      radius = 12.0;
8      area = PI * radius * radius;
9      printf("Area = %f", area);
10 }
```

- `#include <stdio.h>` → direttiva per il preprocessore deputata a includere integralmente nel sorgente il contenuto del file header standard, rendendo disponibili i prototipi delle funzioni di I/O.
- `#define PI 3.14159` → direttiva per la sostituzione testuale; ogni occorrenza dell’etichetta nel codice sottostante viene sostituita letteralmente con il valore definito prima della compilazione.
- `int main(void)` → funzione principale obbligatoria; ogni applicativo C deve presentare una e una sola funzione `main()`, punto di avvio del codice utente al completamento del setup del CRT.

Nelle applicazioni embedded su architetture ARM, le funzioni di libreria standard per la stampa a video (come `printf()`) possono operare mediante la modalità di **semihosting**. Il semihosting è un meccanismo hardware/software che permette al firmware in esecuzione sul microcontrollore di impiegare i canali di input/output della macchina host che ospita il debugger; per realizzare la console/terminale viene impiegata direttamente una delle linee di debug hardware, evitando l’allocazione e l’occupazione di una linea seriale fisica del microcontrollore.

Il linguaggio C supporta due tipologie di **commenti**:

- Commenti a blocco: delimitati all’apertura da `/*` e in chiusura da `*/`, possono estendersi su più righe di testo.
- Commenti a singola linea: introdotti dalla sequenza `//`, estendono il commento fino al termine della riga corrente.

Gli **identificatori** definiscono i nomi univoci attribuiti a elementi quali variabili, funzioni o array per referenziarli prescindendo dall’effettiva locazione di memoria. Le regole sintattiche stabiliscono che:

- Gli identificatori devono essere costituiti da sequenze di caratteri.
- Il primo carattere non deve essere un numero.
- È fatto divieto assoluto di includere caratteri di spaziatura.
- La sintassi è rigorosamente case sensitive (es. `myVar` e `MyVar` identificano due entità differenti).

Lo standard ANSI C definisce un insieme di parole chiave riservate (*keywords*) che possiedono un significato sintattico specifico per il compilatore e non possono in alcun caso essere impiegate come identificatori:

- **auto**: dichiara una variabile locale con allocazione e durata automatica (comportamento predefinito nelle funzioni).

- **break**: interrompe l'esecuzione del ciclo (**for**, **while**, **do-while**) o del blocco **switch** corrente.
- **case**: definisce una specifica etichetta di valore all'interno di un'istruzione **switch**.
- **char**: tipo di dato fondamentale per rappresentare singoli caratteri o interi su 1 byte.
- **const**: qualifica una variabile come di sola lettura, impedendone la modifica successiva.
- **continue**: salta il resto dell'iterazione corrente di un ciclo e passa alla successiva.
- **default**: definisce il ramo di fallback in uno **switch** nel caso nessun **case** corrisponda.
- **do**: avvia un ciclo a controllo post-condizionale in coppia con **while** (eseguito almeno una volta).
- **double**: tipo di dato per numeri in virgola mobile a doppia precisione (64 bit).
- **else**: definisce il blocco alternativo da eseguire quando la condizione dell'**if** risulta falsa.
- **enum**: definisce un tipo di dato per enumerazione, associando identificatori a costanti intere.
- **extern**: dichiara una variabile o funzione la cui definizione effettiva si trova in un'altra unità di traduzione.
- **float**: tipo di dato per numeri in virgola mobile a singola precisione (32 bit).
- **for**: struttura di controllo per cicli composti da inizializzazione, condizione di permanenza e passo di incremento.
- **goto**: esegue un salto incondizionato verso un'etichetta (*label*) definita all'interno della stessa funzione.
- **if**: costrutto condizionale che esegue un blocco di istruzioni solo se l'espressione di test è vera.
- **int**: tipo di dato fondamentale per la rappresentazione di numeri interi con segno.
- **long**: modificatore o tipo di dato per rappresentare interi (o float) con intervallo di valori esteso.
- **register**: suggerisce al compilatore di allocare la variabile direttamente in un registro CPU per minimizzare i tempi di accesso.
- **return**: termina l'esecuzione della funzione corrente e restituisce un eventuale valore al chiamante.
- **short**: modificatore o tipo di dato per interi di dimensione e intervallo ridotti rispetto a **int**.
- **signed**: specifica esplicitamente che un tipo intero memorizza sia valori positivi che negativi.
- **sizeof**: operatore valutato a tempo di compilazione che restituisce la dimensione in byte di un tipo o variabile.
- **static**: mantiene persistente il valore di variabili locali tra chiamate successive, oppure limita lo *scope* di variabili/funzioni globali al singolo modulo.
- **struct**: definisce un tipo di dato eterogeneo composto da più membri memorizzati contigualmente.
- **switch**: costrutto di selezione multipla che devia il flusso in base al valore intero di un'espressione.
- **typedef**: definisce un alias o sinonimo per un tipo di dato preesistente.
- **union**: definisce una struttura dati in cui tutti i membri condividono la medesima area di memoria sovrapposta.
- **unsigned**: specifica che un tipo intero gestisce unicamente valori non negativi (≥ 0).

- **void**: indica un tipo nullo o vuoto, usato per funzioni senza valore di ritorno o puntatori generici privi di tipo tipizzato.
- **volatile**: segnala al compilatore che il valore può variare in modo asincrono (es. registri mappati su hardware o routine ISR), inibendo l'ottimizzazione sui registri.
- **while**: costrutto iterativo a controllo pre-condizionale che reitera finché l'espressione si mantiene vera.

2.2.1 Tipizzazione dei dati e dichiarazione delle variabili

Una **variabile** costituisce l'associazione tra un tipo di dato e un identificatore, atta a rappresentare una o più locazioni di memoria destinate a contenere le informazioni del programma. Nel linguaggio C ogni variabile deve essere dichiarata prima del suo utilizzo.

Il tipo di dato assolve a due compiti distinti:

- Specifica al linker il quantitativo di memoria da riservare per la memorizzazione della variabile.
- Specifica al compilatore le regole di manipolazione e le operazioni lecite sul dato memorizzato.

I tipi di dato fondamentali e la loro occupazione di memoria sono:

- **char** → carattere singolo, dimensione pari a 8 bit.
- **int** → valore intero, dimensione pari a 32 bit.
- **float** → numero in virgola mobile a singola precisione, dimensione pari a 32 bit.
- **double** → numero in virgola mobile a doppia precisione, dimensione pari a 64 bit.

I tipi fondamentali possono essere modificati tramite i prefissi **qualificatori**: **signed**, **unsigned**, **short** e **long**. Quando il tipo è modificato da un qualificatore, la specificazione del termine **int** diviene facoltativa (es. **unsigned** equivale a **unsigned int**).

La classificazione dei tipi interi qualificati definisce i seguenti intervalli numerici e dimensioni:

- **char**, **signed char** → 8 bit, intervallo da -128 a 127 .
- **unsigned char** → 8 bit, intervallo da 0 a 255 .
- **short int**, **signed short int** → 16 bit, intervallo da -32768 a 32767 .
- **unsigned short int** → 16 bit, intervallo da 0 a 65535 .
- **int**, **signed int** → 32 bit, intervallo da -2^{31} a $2^{31} - 1$.
- **unsigned int** → 32 bit, intervallo da 0 a $2^{32} - 1$.
- **long int**, **signed long int** → 32 bit, intervallo da -2^{31} a $2^{31} - 1$.
- **unsigned long int** → 32 bit, intervallo da 0 a $2^{32} - 1$.
- **long long int**, **signed long long int** → 64 bit, intervallo da -2^{63} a $2^{63} - 1$.
- **unsigned long long int** → 64 bit, intervallo da 0 a $2^{64} - 1$.

Per le tipologie in virgola mobile valgono le seguenti estensioni e valori assoluti minimi e massimi:

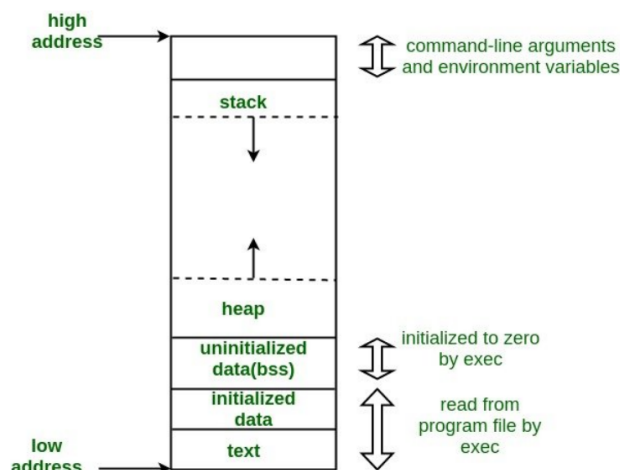
- **float** → 32 bit, minimo assoluto $\pm \sim 10^{-44.85}$, massimo assoluto $\pm \sim 10^{38.53}$.
- **double** → 64 bit, minimo assoluto $\pm \sim 10^{-323.3}$, massimo assoluto $\pm \sim 10^{308.3}$.
- **long double** → 64 bit, minimo assoluto $\pm \sim 10^{-323.3}$, massimo assoluto $\pm \sim 10^{308.3}$.

Sotto il profilo sintattico, si distinguono due **modalità di dichiarazione**:

- **Dichiarazione** → viene associato il tipo all'identificatore senza assegnazione contestuale di valore (es. `int x, y, z;`, `float warpFactor;`, `char text_buffer[10];`, `unsigned index;`).
- **Dichiarazione con definizione** → contestualmente alla dichiarazione viene assegnato un valore esplicito mediante l'operatore di assegnazione = (es. `unsigned y = 12;`, `long int myVar = 0x12345678;`, `char first = 'a', second, third = 'c';`, in cui solo `first` e `third` risultano inizializzate).

2.3 Organizzazione della memoria e classi di memorizzazione

Lo spazio di indirizzamento della memoria è organizzato in segmenti funzionali distinti, disposti convenzionalmente dagli indirizzi bassi (*low address*) verso gli indirizzi alti (*high address*):



- **Code Segment (Text Segment)** → contiene le istruzioni binarie del programma eseguite dal processore. È un'area ad accesso esclusivo di sola lettura (*read-only*); nei microcontrollori risiede permanentemente nella memoria non volatile (Flash).
- **Data Segment** → accoglie le variabili globali e le variabili statiche, suddiviso internamente in due sezioni:
 - **Initialized Data** → ospita variabili globali e statiche inizializzate esplicitamente a un valore diverso da zero. Si ripartisce a sua volta in una *read-only area* (costanti e string literal) e una *read-write area*. Nei microcontrollori i valori iniziali risiedono fisicamente in memoria Flash e vengono trasferiti in RAM dallo startup code durante la sequenza di reset, consentendone la modifica a run-time.
 - **Uninitialized Data (BSS, Block Started by Symbol)** → ospita le variabili globali e statiche non inizializzate o inizializzate esplicitamente a zero; viene azzerato in memoria RAM dal codice di avvio (`csstartup`) prima dell'avvio dell'applicazione.
- **Heap** → destinato all'allocazione dinamica della memoria a run-time mediante le funzioni `malloc()` e `free()`; è allocato in RAM e presenta una direzione di crescita verso gli indirizzi alti.
- **Stack** → operante in RAM secondo logica LIFO (*Last In - First Out*), alloca le variabili locali, le variabili allocate nei registri e l'intero contesto di esecuzione delle funzioni (parametri e indirizzi di ritorno); presenta una direzione di crescita verso il basso (indirizzi decrescenti). Un dimensionamento non idoneo o un eccessivo riempimento dello stack può generare uno sfondamento delle aree contigue fino a raggiungere il Code Segment, costituendo una nota vulnerabilità software.
- **Argomenti da riga di comando e variabili d'ambiente** → risiedono al vertice superiore della mappa di memoria, in corrispondenza degli indirizzi più elevati.

Le classi di memorizzazione (**storage classes**) determinano il segmento di allocazione, il valore di inizializzazione predefinito, la visibilità (*scope*) e la persistenza temporale (*life*) delle variabili:

- **auto** → allocata nello Stack; assume valore iniziale indefinito (*garbage*); visibilità limitata all'interno del blocco di definizione (*within block*); tempo di vita limitato alla terminazione del blocco (*end of block*).
- **extern** → allocata nel Data Segment; valore iniziale predefinito pari a zero; visibilità globale estesa su molteplici file di progetto; tempo di vita esteso all'intera durata dell'esecuzione del programma.
- **static** → allocata nel Data Segment; valore iniziale predefinito pari a zero; visibilità confinata al blocco locale in cui è dichiarata (*within block*); tempo di vita permanente per l'intera durata del programma.
- **register** → allocata direttamente nei registri interni della CPU; assume valore iniziale indefinito (*garbage*); visibilità limitata al blocco (*within block*); tempo di vita limitato alla fine del blocco.

Storage classes in C				
Storage Specifier	Storage	Initial value	Scope	Life
auto	stack	Garbage	Within block	End of block
extern	Data segment	Zero	global Multiple files	Till end of program
static	Data segment	Zero	Within block	Till end of program
register	CPU Register	Garbage	Within block	End of block

2.4 Puntatori

All'interno di un'applicazione in linguaggio C per sistemi embedded, l'accesso alle variabili può avvenire sia mediante identificatore diretto, leggendone e modificandone il valore memorizzato, sia operando direttamente sull'indirizzo fisico occupato dalla variabile nello spazio di memoria.

Un **puntatore** è una variabile destinata a memorizzare l'indirizzo di memoria di un'altra variabile o di una funzione. Tale meccanismo realizza una modalità di **indirizzamento indiretto**: tramite il puntatore è possibile accedere e manipolare il valore della cella di memoria puntata e, al contempo, modificare dinamicamente l'indirizzo memorizzato nel puntatore stesso per indirizzare locazioni differenti.

Dichiarazione, Inizializzazione e Dereferenziazione

La sintassi fondamentale per la **definizione** di un puntatore impone l'esplicitazione del tipo di dato a cui il puntatore fa riferimento:

```
type *p;
```

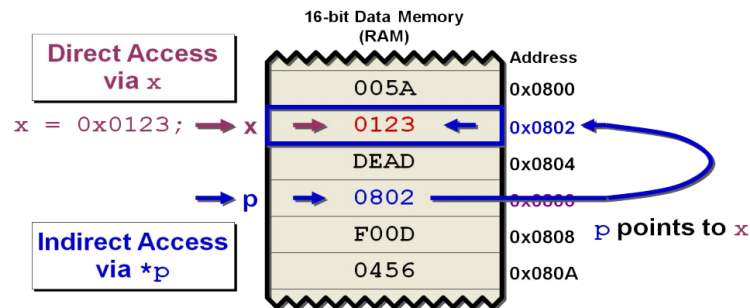
Nell'istruzione di dichiarazione, l'operatore ***** qualifica la variabile **p** come puntatore, mentre **type** identifica il tipo di dato allocato nella locazione bersaglio (**int**, **float**, ecc...).

L'inizializzazione di un puntatore richiede l'**assegnazione dell'indirizzo** di memoria di una variabile esistente dello stesso tipo base. Tale indirizzo viene ottenuto antepo-
nendo al nome della

variabile l'operatore unario di indirizzo `&` (*address of*). Poiché l'assegnamento imposta l'indirizzo contenuto nel puntatore e non il valore nella locazione puntata, l'operatore `*` non deve essere adoperato:

```
1 p = &x; // Assegna al puntatore p l'indirizzo della variabile x
```

Una volta inizializzato il puntatore all'indirizzo di una variabile, esso consente di operare direttamente sulla variabile puntata mediante l'operatore unario di **dereferenziazione** `*`. A differenza del suo utilizzo in fase di dichiarazione, all'interno di un'espressione il simbolo `*` assume il significato di "accedi al valore della variabile puntata da `p`" invece che all'indirizzo contenuto in `p` stesso. Qualora `p` punti a `x`, l'istruzione `y = *p;` produce un effetto equivalente all'assegnamento diretto `y = x;`.



Puntatori costanti e puntatori variabili

Nel linguaggio C i concetti di indirizzo e di puntatore sono strettamente equivalenti. L'espressione `&x` rappresenta un puntatore alla variabile `x`. Poiché la variabile `x` mantiene una collocazione fissa nello spazio di memoria determinata dal compilatore e dal linker, l'indirizzo `&x` non subisce traslazioni a run-time e costituisce a tutti gli effetti un **puntatore costante**.

Al contrario nei **puntatori variabili** l'indirizzo di destinazione può essere riassegnato a run-time per indirizzare variabili differenti in qualsiasi punto del codice. L'accesso tramite dereferenziazione opererà sempre sulla locazione correntemente memorizzata nel puntatore.

```
1 int a, b;
2 int *p;
3
4 p = &a; //punto la variabile a
5 *p = 10; //assegnazione indiretta alla variabile a
6
7 p = &b //cambio variabile da puntare
8 *p = 5; //assegnazione indiretta alla variabile b
```

Interfacciamento con gli array

Gli elementi costituenti un array vengono memorizzati dal compilatore in posizioni fisicamente contigue e sequenziali all'interno dello spazio di memoria. Tale caratteristica architetturale consente di scorrere e indicizzare gli elementi sia mediante la classica notazione con parentesi quadre sia sfruttando l'indirizzo base del primo elemento e l'aritmetica dei puntatori.

Dato un array e un puntatore compatibile:

```
1 int x[5] = {1, 2, 3, 4, 5};
2 int *p;
```

L'indirizzamento iniziale della testa dell'array può essere espresso mediante due forme sintattiche:

- `p = x;` → forma sintattica valida unicamente per gli array; il nome dell'array non indicizzato coincide direttamente con il puntatore al suo primo elemento.

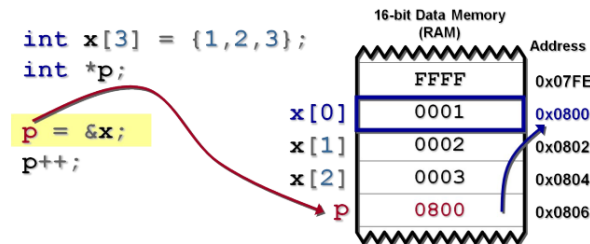
- $p = \&x[0]$; → notazione esplicita che estrae puntualmente l'indirizzo di memoria del primo elemento (indice zero) dell'array.

$p = \&x$; → sintassi lecita solo per singole variabili scalari.

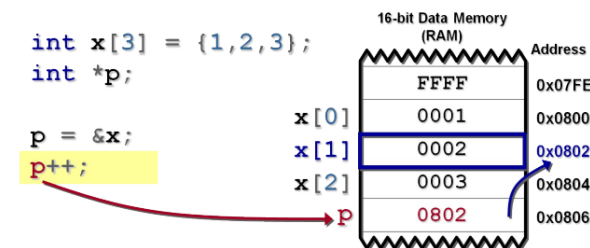
Aritmetica dei puntatori

L'aritmetica dei puntatori consente di traslare l'indirizzo contenuto in un puntatore mediante operazioni di incremento, decremento e somma/sottrazione di costanti intere. La caratteristica cardine risiede nel fatto che l'incremento di un puntatore sposta il riferimento all'elemento successivo dell'array indipendentemente dall'ampiezza fisica in byte occupata dal tipo di dato.

First, we **set the pointer to the address of the first element** (which is the address of the array itself)



Next, we **increment the pointer** so that it now points to the next element in the array.



La regola fondamentale stabilisce che *ogni operazione di incremento o decremento modifica l'indirizzo effettivo di un numero di byte pari a quello occupato dal tipo puntato*. Se il tipo è a 32 bit, l'incremento del puntatore aggiunge 4 byte all'indirizzo in esso contenuto.

L'associazione dell'operatore di dereferenziazione $*$ con l'operatore di post-incremento $++$ è fonte di ambiguità sintattiche. Nel linguaggio C entrambi gli operatori possiedono il medesimo livello di precedenza; tuttavia, la loro **associatività opera da destra a sinistra** (*right-to-left associativity*). Tale proprietà conferisce all'operatore postfix $++$ una precedenza effettiva superiore rispetto all'operatore di dereferenziazione $*$.

Si distinguono due costrutti sintattici fondamentali:

- **Post-incremento del puntatore** ($p++$, $*p++$, $*(p++)$) → l'operatore di post-incremento agisce sulla variabile puntatore p . L'istruzione $z = *(p++)$; del tutto equivalente a $z = *p++$; opera la sequenza:

$$z = *p; \quad p = p + 1;$$

Viene prima valutato il valore presente nella cella di memoria puntata da p (assegnato alla variabile z), e successivamente viene incrementato l'indirizzo contenuto nel puntatore p . Il contenuto della cella in memoria non subisce modifiche.

- **Post-incremento del dato puntato** ($(*p)++$) → la presenza delle parentesi tonde impone la priorità all'operazione di dereferenziazione prima del post-incremento. L'istruzione $z = (*p)++$; opera la sequenza:

$$z = *p; \quad *p = *p + 1;$$

Viene prima estratto il valore puntato da `p` per assegnarlo a `z`, dopodiché viene incrementato di un'unità il valore numerico contenuto all'interno della cella di memoria puntata da `p`. L'indirizzo memorizzato nel puntatore `p` rimane inalterato.

Puntatori a funzione

Nel linguaggio C i puntatori possono essere impiegati per memorizzare gli indirizzi del codice eseguibile, puntando al punto di ingresso delle funzioni. Questa caratteristica consente di passare funzioni come argomenti ad altre funzioni, permettendo la realizzazione di architetture software modulari.

La dichiarazione di un puntatore a funzione impone l'inclusione dell'asterisco e dell'identificatore tra parentesi tonde, associando la firma della funzione attesa:

```
1 int (*fp)(int x); // Dichiarazione del puntatore a funzione con nome fp
```

I vincoli di conformità sintattica stabiliti dalla dichiarazione sono:

- La funzione a cui il puntatore fa riferimento deve accettare come argomento esattamente un parametro di tipo `int`.
- La funzione a cui il puntatore fa riferimento deve restituire un valore di tipo intero (`int`).

L'inizializzazione di un puntatore a funzione si esegue assegnando al puntatore il nome della funzione bersaglio:

```
1 int (*fp)(int x); // Puntatore a funzione
2 int foo(int x);   // Prototipo della funzione
3
4 fp = foo;         // Inizializzazione: fp punta alla funzione foo
```

In seguito all'assegnamento, `fp` contiene l'indirizzo di ingresso della funzione `foo`.